

**LAPORAN
MECHINE LEARNING**



Disusun Oleh :

Nama : Muhammad Nabil Fitterzon
NPM : G1A024103
Kelas : A

Dosen Pengampu :

1. Ir. Arie Vatesia, S.T., M.T.I., Ph.D., IPP.

**PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS BENGKULU
2026**

BAB I

PENDAHULUAN

1.1 Latar Belakang

Machine Learning merupakan bagian dari kecerdasan buatan yang digunakan untuk menemukan pola dari data dan menghasilkan prediksi atau pengelompokan. Pada tugas ini dilakukan beberapa percobaan menggunakan Python dan pustaka scikit-learn serta mglearn. Percobaan mencakup algoritma klasifikasi, preprocessing, reduksi dimensi, dan clustering. Dataset yang digunakan berasal dari dataset contoh yang tersedia pada mglearn dan scikit-learn, antara lain forge, breast cancer, LFW people, digits, dan make_moons.

1.2 Tujuan

Tujuan kegiatan ini adalah memahami penerapan beberapa algoritma Machine Learning, melihat hasil prediksi dan pengelompokan, memahami pengaruh preprocessing dan reduksi dimensi, serta melakukan evaluasi hasil clustering menggunakan metrik yang tersedia.

1.3 Lingkup Percobaan

Percobaan meliputi K-Nearest Neighbors (KNN), Logistic Regression, Linear SVC, Naive Bayes, Decision Tree, Random Forest, Support Vector Machine, Multi-Layer Perceptron, MinMax Scaling, PCA, NMF, t-SNE, K-Means, Agglomerative Clustering, dan DBSCAN.

BAB II

METODOLOGI

2.1 Lingkungan Pemrograman

Eksperimen dijalankan pada Google Colab dengan Python. Pustaka mglearn juga dipasang dan digunakan untuk visualisasi serta dataset contoh :

```
import numpy as np
import pandas as pd
import matplotlib
import matplotlib.pyplot as plt
import scipy
import sklearn

print("NumPy:", np.__version__)
print("Pandas:", pd.__version__)
print("Matplotlib:", matplotlib.__version__)
print("SciPy:", scipy.__version__)
print("Scikit-learn:", sklearn.__version__)
```

Kode 2.1 Pemeriksaan versi pustaka

Output menunjukkan versi pustaka yang digunakan pada saat percobaan.

BAB III

HASIL DAN PEMBAHASAN

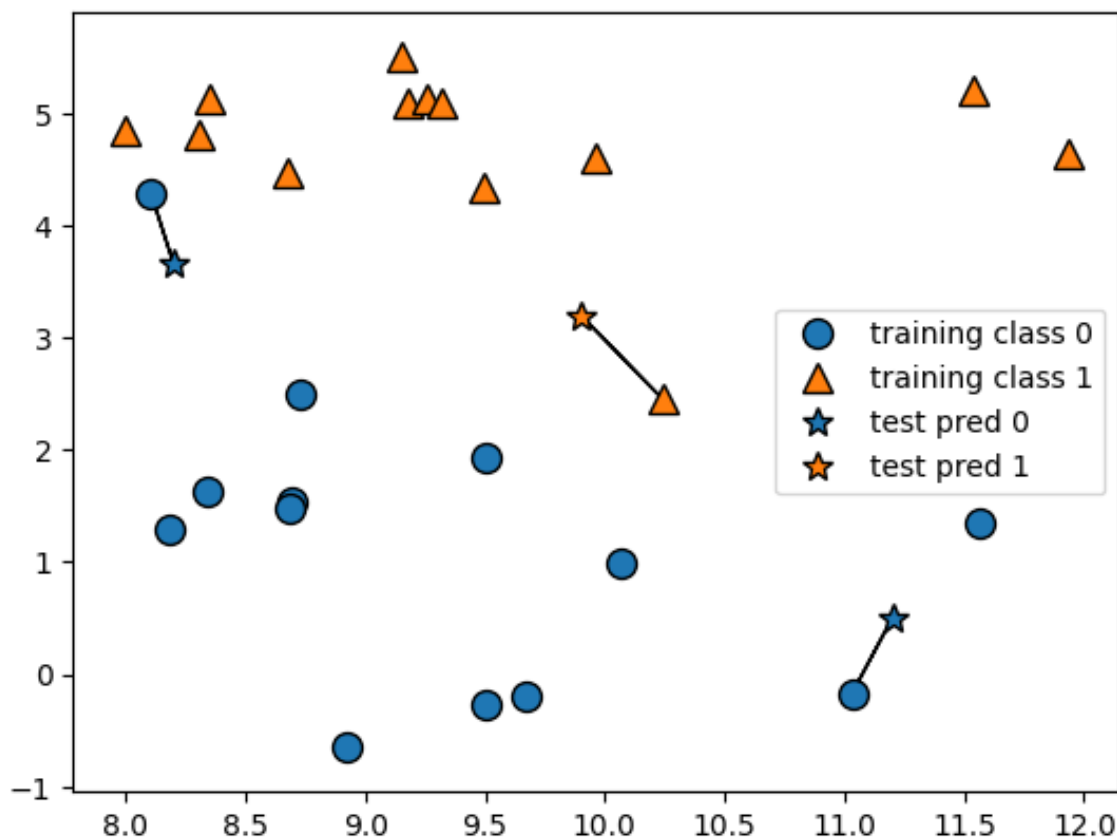
3.1 K-Nearest Neighbors (KNN)

Percobaan KNN menggunakan dataset forge dan membagi data menjadi data training dan testing. Model dibuat dengan tiga tetangga ($n_neighbors=3$). Hasil prediksi pada data pengujian adalah [1, 0, 1, 0, 1, 0, 0] dengan akurasi 0,8571 atau 85,71%. Hasil ini menunjukkan bahwa pada data pengujian yang digunakan, sebagian besar sampel berhasil diklasifikasikan dengan benar. Visualisasi KNN digunakan untuk melihat hubungan posisi data terhadap kelasnya.

Kode percobaan (Cell 2):

```
import mglearn
import matplotlib.pyplot as plt

mglearn.plots.plot_knn_classification(n_neighbors=1)
plt.show()
```



Gambar hasil percobaan pada Cell 2.

Kode percobaan (Cell 3):

```
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
```

```

import mglearn

X, y = mglearn.datasets.make_forge()

X_train, X_test, y_train, y_test = train_test_split(
    X, y, random_state=0
)

clf = KNeighborsClassifier(n_neighbors=3)

clf.fit(X_train, y_train)

print("Prediksi:", clf.predict(X_test))

print("Akurasi:", clf.score(X_test, y_test))

```

Output:

```

Prediksi: [1 0 1 0 1 0 0]
Akurasi: 0.8571428571428571

```

3.2 Logistic Regression dan Linear SVC

Logistic Regression dan Linear SVC digunakan untuk melihat pembentukan batas keputusan pada data dua dimensi. Kedua model dilatih pada dataset forge dan divisualisasikan menggunakan decision boundary. Visualisasi membantu melihat bagaimana masing-masing model memisahkan dua kelas berdasarkan fitur yang tersedia.

Kode percobaan (Cell 4):

```

from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC

X, y = mglearn.datasets.make_forge()

fig, axes = plt.subplots(1, 2, figsize=(10, 3))

for model, ax in zip([LinearSVC(), LogisticRegression()], axes):
    clf = model.fit(X, y)

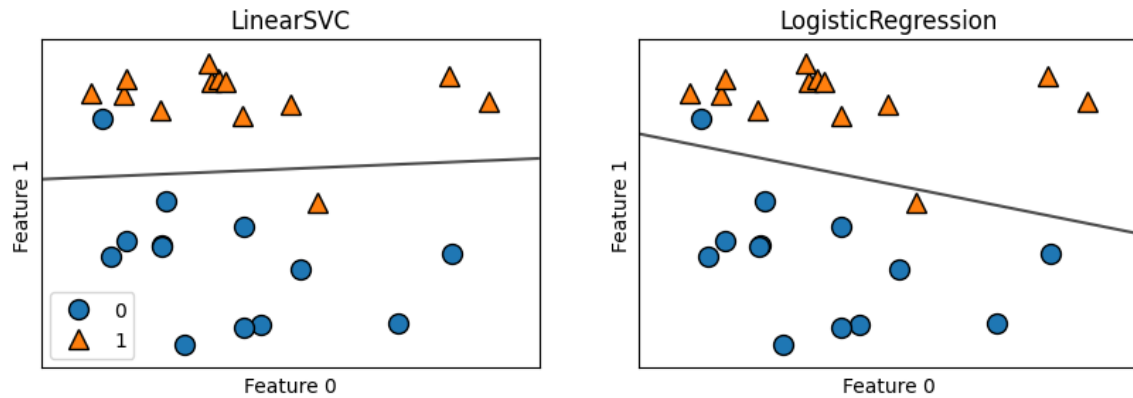
    mglearn.plots.plot_2d_separator(
        clf, X, fill=False, eps=0.5,
        ax=ax, alpha=.7
    )

    mglearn.discrete_scatter(X[:, 0], X[:, 1], y, ax=ax)

    ax.set_title("{}".format(clf.__class__.__name__))
    ax.set_xlabel("Feature 0")
    ax.set_ylabel("Feature 1")

axes[0].legend()
plt.show()

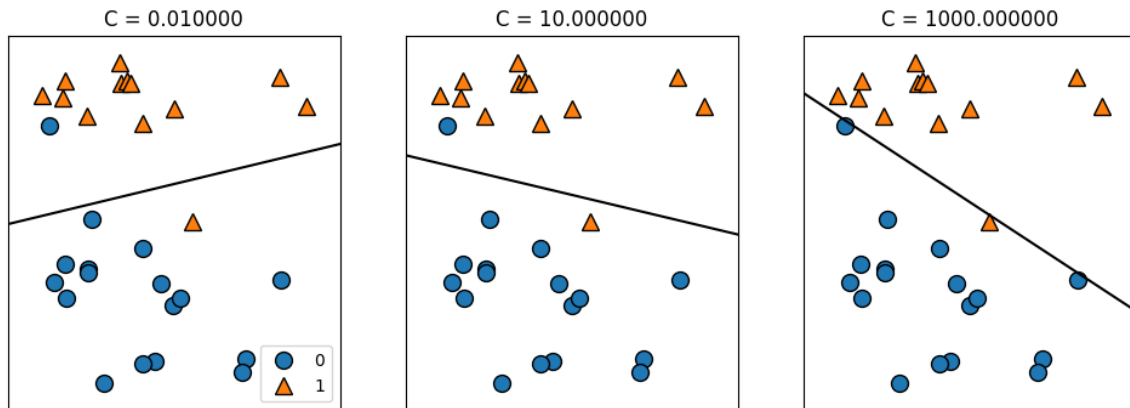
```



Gambar hasil percobaan pada Cell 4.

Kode percobaan (Cell 5):

```
mglearn.plots.plot_linear_svc_regularization()
plt.show()
```



Gambar hasil percobaan pada Cell 5.

3.3 Naive Bayes

Gaussian Naive Bayes diuji pada dataset forge dengan pembagian data training dan testing. Prediksi yang diperoleh adalah [1, 0, 1, 0, 1, 0, 0] dan akurasi sebesar 0,8571 atau 85,71%. Hasil tersebut menunjukkan bahwa model dapat mengklasifikasikan sebagian besar data pengujian dengan benar pada dataset yang digunakan.

Kode percobaan (Cell 6):

```
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import train_test_split
import mglearn

X, y = mglearn.datasets.make_forge()

X_train, X_test, y_train, y_test = train_test_split(
    X, y, random_state=0
)
```

```
clf = GaussianNB()
clf.fit(X_train, y_train)

print("Prediksi:", clf.predict(X_test))
print("Akurasi:", clf.score(X_test, y_test))
```

Output:

```
Prediksi: [1 0 1 0 1 0 0]
Akurasi: 0.8571428571428571
```

3.4 Decision Tree

Decision Tree dilatih pada dataset forge. Nilai akurasi yang tercatat adalah 1,0 atau 100%. Namun, kode pada percobaan ini menghitung skor menggunakan X dan y keseluruhan, bukan data pengujian terpisah. Oleh karena itu, angka 100% pada percobaan ini menunjukkan ketepatan model terhadap data yang digunakan untuk penilaian tersebut dan tidak boleh disebut sebagai akurasi data test.

Kode percobaan (Cell 7):

```
from sklearn.tree import DecisionTreeClassifier
import mglearn
import matplotlib.pyplot as plt

X, y = mglearn.datasets.make_forge()

tree = DecisionTreeClassifier(random_state=0)
tree.fit(X, y)

print("Akurasi:", tree.score(X, y))
```

Output:

```
Akurasi: 1.0
```

Kode percobaan (Cell 8):

```
from sklearn.tree import export_graphviz
import graphviz

export_graphviz(
    tree,
    out_file="tree.dot",
    class_names=["class 0", "class 1"],
    feature_names=["feature 0", "feature 1"],
    impurity=False,
    filled=True
)

with open("tree.dot") as f:
    dot_graph = f.read()

graphviz.Source(dot_graph)
```

Output:

```
<graphviz.sources.Source at 0x7e454fa974d0>
```

3.5 Random Forest

Random Forest dibuat menggunakan lima decision tree (`n_estimators=5`). Pada data pengujian, model menghasilkan prediksi `[1, 0, 0, 0, 1, 0, 0]` dengan akurasi 0,7143 atau 71,43%. Visualisasi memperlihatkan pembagian wilayah keputusan dari beberapa tree dan hasil gabungannya.

Kode percobaan (Cell 9):

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
import mglearn

X, y = mglearn.datasets.make_forge()

X_train, X_test, y_train, y_test = train_test_split(
    X, y, random_state=0
)

forest = RandomForestClassifier(
    n_estimators=5,
    random_state=2
)

forest.fit(X_train, y_train)

print("Prediksi:", forest.predict(X_test))
print("Akurasi:", forest.score(X_test, y_test))
```

Output:

```
Prediksi: [1 0 0 0 1 0 0]
Akurasi: 0.7142857142857143
```

Kode percobaan (Cell 10):

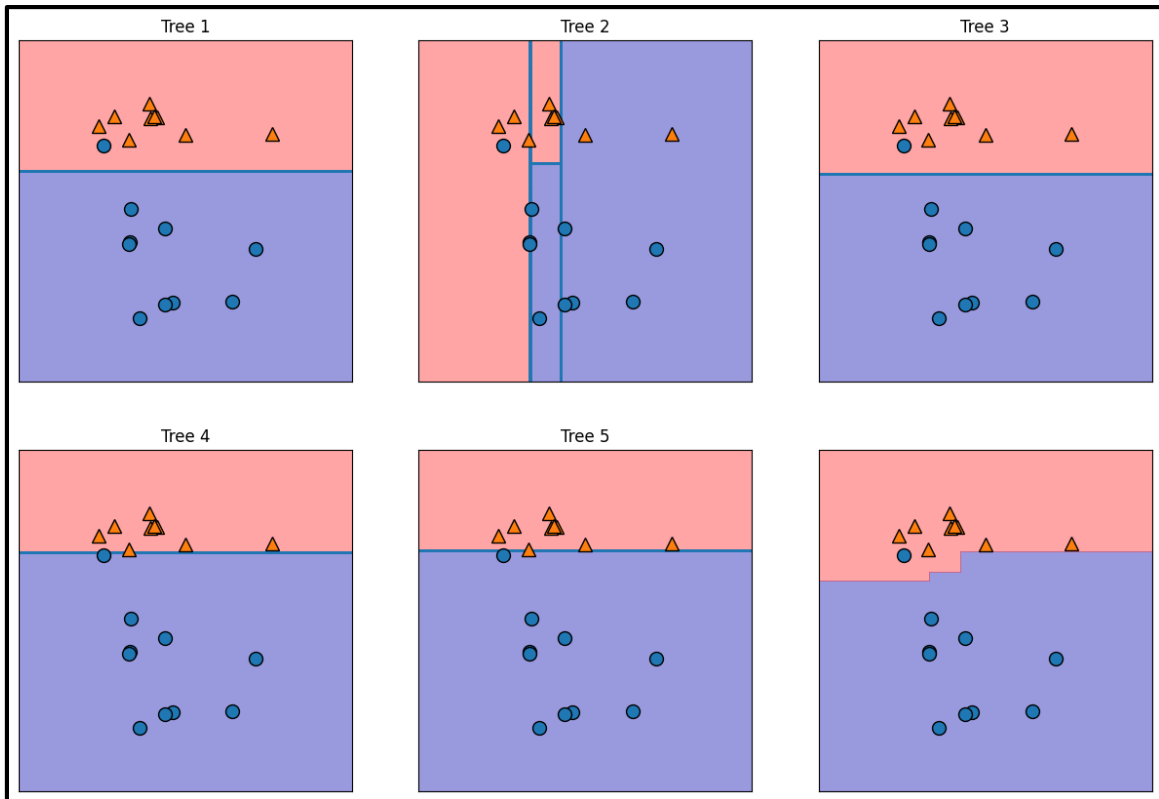
```
import matplotlib.pyplot as plt

fig, axes = plt.subplots(2, 3, figsize=(15, 10))

for i, (ax, tree) in enumerate(
    zip(axes.ravel(), forest.estimators_)
):
    ax.set_title("Tree {}".format(i + 1))
    mglearn.plots.plot_tree_partition(
        X_train, y_train, tree, ax=ax
    )

mglearn.plots.plot_2d_separator(
    forest, X_train, fill=True, ax=axes[-1, -1], alpha=.4
)
mglearn.discrete_scatter(
    X_train[:, 0], X_train[:, 1], y_train,
    ax=axes[-1, -1]
)

plt.show()
```



Gambar hasil percobaan pada Cell 10.

3.6 Support Vector Machine (SVM)

SVM dengan kernel RBF digunakan pada dataset yang dibuat melalui `make_handcrafted_dataset()`. Visualisasi menampilkan batas keputusan dan support vectors. Support vectors merupakan titik data yang berada dekat dengan batas pemisah dan berperan penting dalam pembentukan keputusan model.

Kode percobaan (Cell 11):

```
from sklearn.svm import SVC
import mglearn
import matplotlib.pyplot as plt

X, y = mglearn.tools.make_handcrafted_dataset()

svm = SVC(kernel='rbf', C=10, gamma=0.1).fit(X, y)

mglearn.plots.plot_2d_separator(svm, X, eps=.5)

mglearn.discrete_scatter(X[:, 0], X[:, 1], y)

# Menampilkan support vectors
sv = svm.support_vectors_

sv_labels = svm.dual_coef_.ravel() > 0

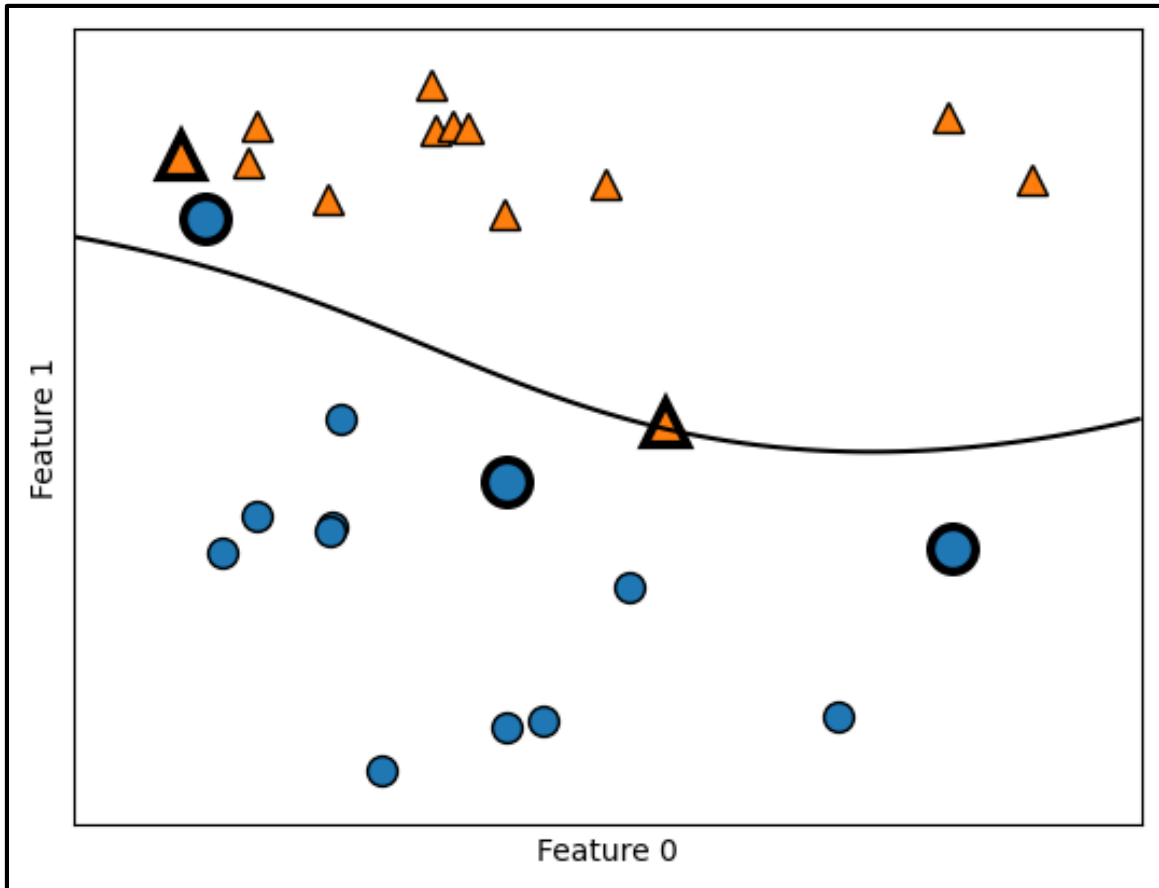
mglearn.discrete_scatter(
```

```

sv[:, 0], sv[:, 1],
sv_labels,
s=15,
markeredgewidth=3
)

plt.xlabel("Feature 0")
plt.ylabel("Feature 1")
plt.show()

```



Gambar hasil percobaan pada Cell 11.

3.7 Multi-Layer Perceptron (MLP)

MLPClassifier dengan solver lbfgs digunakan pada dataset forge. Prediksi yang diperoleh adalah [1, 0, 1, 0, 1, 0, 0] dengan akurasi 0,8571 atau 85,71%. Grafik decision boundary menunjukkan pemisahan kelas yang dihasilkan oleh jaringan saraf pada data dua dimensi.

Kode percobaan (Cell 12):

```

from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import train_test_split
import mglearn

X, y = mglearn.datasets.make_forge()

X_train, X_test, y_train, y_test = train_test_split(
    X, y, random_state=0
)

```

```

)

mlp = MLPClassifier(
    solver='lbfgs',
    random_state=0
)

mlp.fit(X_train, y_train)

print("Prediksi:", mlp.predict(X_test))
print("Akurasi:", mlp.score(X_test, y_test))

```

Output:

Prediksi: [1 0 1 0 1 0 0]
 Akurasi: 0.8571428571428571

Kode percobaan (Cell 13):

```

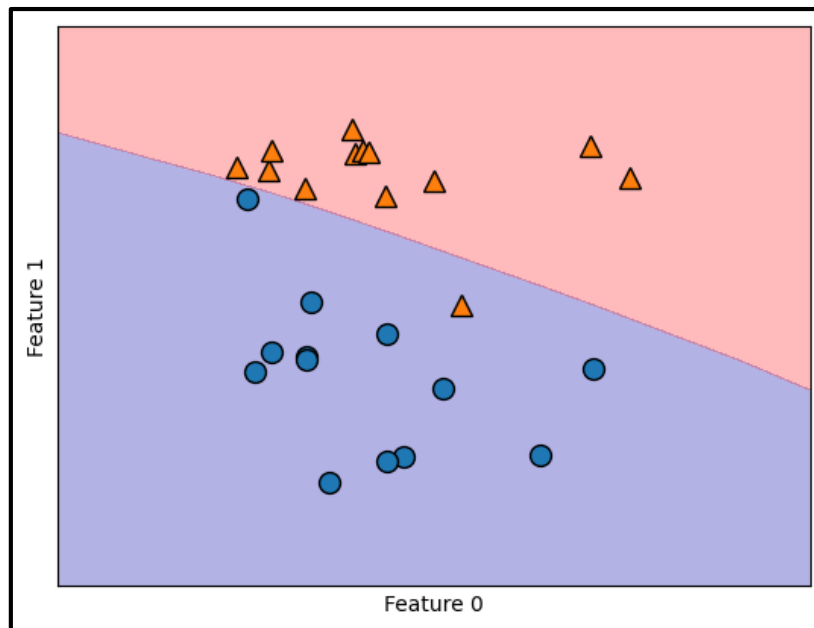
import matplotlib.pyplot as plt

mglearn.plots.plot_2d_separator(
    mlp, X, fill=True, alpha=.3
)

mglearn.discrete_scatter(
    X[:, 0], X[:, 1], y
)

plt.xlabel("Feature 0")
plt.ylabel("Feature 1")
plt.show()

```



Gambar hasil percobaan pada Cell 13.

3.8 MinMax Scaling

Pada dataset breast cancer dilakukan pembagian data menjadi 426 data training dan 143 data testing, dengan 30 fitur. MinMaxScaler kemudian dipasang pada data training dan digunakan

untuk melakukan transformasi. Hasil output menunjukkan bahwa nilai minimum setelah scaling menjadi 0 dan nilai maksimum menjadi 1. Dengan demikian, fitur yang memiliki rentang nilai berbeda dapat dinormalisasi ke rentang yang sama.

Kode percobaan (Cell 14):

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler

# load dan split dataset
cancer = load_breast_cancer()

X_train, X_test, y_train, y_test = train_test_split(
    cancer.data,
    cancer.target,
    random_state=1
)

print("Training set:", X_train.shape)
print("Test set:", X_test.shape)

scaler = MinMaxScaler()

scaler.fit(X_train)

X_train_scaled = scaler.transform(X_train)

print("\nBentuk data setelah scaling:", X_train_scaled.shape)
print("\nMinimum sebelum scaling:")
print(X_train.min(axis=0))

print("\nMaksimum sebelum scaling:")
print(X_train.max(axis=0))

print("\nMinimum setelah scaling:")
print(X_train_scaled.min(axis=0))

print("\nMaksimum setelah scaling:")
print(X_train_scaled.max(axis=0))
```

Output:

```
Training set: (426, 30)
Test set: (143, 30)
```

```
Bentuk data setelah scaling: (426, 30)
```

```
Minimum sebelum scaling:
```

```
[6.981e+00 9.710e+00 4.379e+01 1.435e+02 5.263e-02 1.938e-02 0.000e+00
0.000e+00 1.060e-01 5.024e-02 1.153e-01 3.602e-01 7.570e-01 6.802e+00
1.713e-03 2.252e-03 0.000e+00 0.000e+00 9.539e-03 8.948e-04 7.930e+00
1.202e+01 5.041e+01 1.852e+02 7.117e-02 2.729e-02 0.000e+00 0.000e+00
1.566e-01 5.521e-02]
```

```
Maksimum sebelum scaling:
```

```
[2.811e+01 3.928e+01 1.885e+02 2.501e+03 1.634e-01 2.867e-01 4.268e-01
2.012e-01 3.040e-01 9.575e-02 2.873e+00 4.885e+00 2.198e+01 5.422e+02
3.113e-02 1.354e-01 3.960e-01 5.279e-02 6.146e-02 2.984e-02 3.604e+01]
```

```
4.954e+01 2.512e+02 4.254e+03 2.226e-01 9.379e-01 1.170e+00 2.910e-01
5.774e-01 1.486e-01]
```

Minimum setelah scaling:

```
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
0. 0. 0. 0. 0. 0.]
```

Maksimum setelah scaling:

```
[1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1.
1. 1. 1. 1. 1. 1.]
```

3.9 Principal Component Analysis (PCA)

PCA diterapkan pada dataset breast cancer setelah standardisasi. Data yang semula berukuran 569 sampel dengan 30 fitur direduksi menjadi dua komponen utama. Explained variance ratio untuk komponen pertama adalah 0,44272026 dan komponen kedua 0,18971182. Jika dijumlahkan, kedua komponen menjelaskan sekitar 63,24% variasi data. Grafik PCA digunakan untuk melihat penyebaran data pada dua komponen utama.

Kode percobaan (Cell 15):

```
from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

cancer = load_breast_cancer()

X_scaled = StandardScaler().fit_transform(cancer.data)

pca = PCA(n_components=2)

X_pca = pca.fit_transform(X_scaled)

print("Bentuk data sebelum PCA:", cancer.data.shape)
print("Bentuk data setelah PCA:", X_pca.shape)
print("Explained variance ratio:")
print(pca.explained_variance_ratio_)
```

Output:

```
Bentuk data sebelum PCA: (569, 30)
Bentuk data setelah PCA: (569, 2)
Explained variance ratio:
[0.44272026 0.18971182]
```

3.10 Non-negative Matrix Factorization (NMF)

Dataset LFW people digunakan untuk percobaan NMF. Data dibagi menjadi 2267 data training dan 756 data testing, dengan bentuk citra 87×65 . NMF menggunakan 15 komponen dan menghasilkan data training berukuran 2267×15 serta data testing 756×15 . Komponen-komponen tersebut divisualisasikan sebagai citra untuk melihat representasi fitur yang dipelajari.

Kode percobaan (Cell 17):

```

from sklearn.datasets import fetch_lfw_people
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt

# Load dataset wajah
people = fetch_lfw_people(
    min_faces_per_person=20,
    resize=0.7
)

image_shape = people.images[0].shape

# Ubah gambar menjadi data 2D
X_people = people.data
y_people = people.target

# Bagi data training dan test
X_train, X_test, y_train, y_test = train_test_split(
    X_people,
    y_people,
    stratify=y_people,
    random_state=0
)

print("X_train:", X_train.shape)
print("X_test:", X_test.shape)
print("Image shape:", image_shape)

```

Output:

```

X_train: (2267, 5655)
X_test: (756, 5655)
Image shape: (87, 65)

```

Kode percobaan (Cell 18):

```

from sklearn.decomposition import NMF

# Membuat NMF dengan 15 komponen
nmf = NMF(
    n_components=15,
    random_state=0
)

nmf.fit(X_train)

X_train_nmf = nmf.transform(X_train)
X_test_nmf = nmf.transform(X_test)

print("X_train_nmf:", X_train_nmf.shape)
print("X_test_nmf:", X_test_nmf.shape)

```

Output:

```

/usr/local/lib/python3.13/dist-packages/sklearn/decomposition/_nmf.py:1742:
ConvergenceWarning: Maximum number of iterations 200 reached. Increase it to improve
convergence.
  warnings.warn(

X_train_nmf: (2267, 15)
X_test_nmf: (756, 15)

```

3.11 t-SNE

Dataset digits yang memiliki 1797 sampel dan 64 fitur ditransformasikan menggunakan t-SNE menjadi dua dimensi. Hasilnya berukuran 1797×2 . Visualisasi dua dimensi membantu melihat pola kedekatan data berdasarkan struktur lokal yang dipelajari t-SNE.

Kode percobaan (Cell 20):

```
from sklearn.datasets import load_digits
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt

digits = load_digits()

tsne = TSNE(random_state=42)

digits_tsne = tsne.fit_transform(digits.data)

print("Bentuk data sebelum t-SNE:", digits.data.shape)
print("Bentuk data setelah t-SNE:", digits_tsne.shape)
```

Output:

```
Bentuk data sebelum t-SNE: (1797, 64)
Bentuk data setelah t-SNE: (1797, 2)
```

3.12 K-Means Clustering

K-Means digunakan dengan jumlah cluster sebanyak 10 pada data wajah. Dua puluh label cluster pertama ditampilkan sebagai contoh, kemudian jumlah data pada setiap cluster dihitung. Distribusi cluster yang diperoleh adalah [187, 163, 376, 228, 249, 217, 248, 218, 244, 137]. Nilai tersebut menunjukkan bahwa ukuran setiap cluster tidak sama.

Kode percobaan (Cell 22):

```
from sklearn.cluster import KMeans

kmeans = KMeans(n_clusters=10, random_state=0)

kmeans.fit(X_train)

print("Cluster yang terbentuk:")
print(kmeans.labels_[:20])
```

Output:

```
Cluster yang terbentuk:
[8 9 8 3 5 5 7 5 7 6 6 7 8 9 3 6 6 7 8 4]
```

Kode percobaan (Cell 23):

```
import numpy as np

print("Jumlah data pada setiap cluster:")
print(np.bincount(kmeans.labels_))
```

Output:

```
Jumlah data pada setiap cluster:
[187 163 376 228 249 217 248 218 244 137]
```

Kode percobaan (Cell 24):

```
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

pca = PCA(n_components=2, random_state=0)

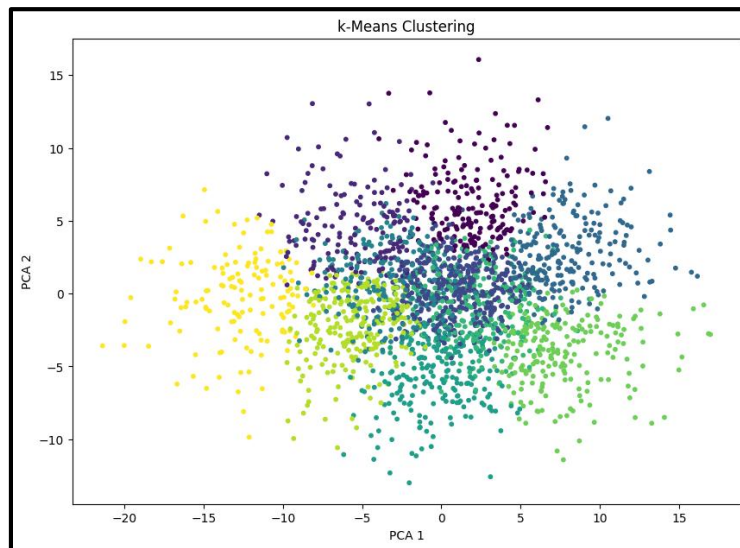
X_train_pca = pca.fit_transform(X_train)

plt.figure(figsize=(10, 7))

plt.scatter(
    X_train_pca[:, 0],
    X_train_pca[:, 1],
    c=kmeans.labels_,
    s=10
)

plt.xlabel("PCA 1")
plt.ylabel("PCA 2")
plt.title("k-Means Clustering")

plt.show()
```



Gambar hasil percobaan pada Cell 24.

3.13 Agglomerative Clustering

Agglomerative Clustering diuji pada dataset make_moons yang terdiri dari 200 sampel. Data terlebih dahulu distandardisasi dan kemudian dikelompokkan menjadi dua cluster. Output menampilkan label cluster dan grafik hasil pengelompokan.

Kode percobaan (Cell 25):

```
from sklearn.datasets import make_moons
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import AgglomerativeClustering
import matplotlib.pyplot as plt
```

```

X, y = make_moons(n_samples=200, noise=0.05, random_state=0)

# Rescale data
scaler = StandardScaler()
scaler.fit(X)
X_scaled = scaler.transform(X)

# Agglomerative Clustering
agg = AgglomerativeClustering(n_clusters=2)

clusters = agg.fit_predict(X_scaled)

print("Cluster hasil Agglomerative:")
print(clusters[:20])

```

Output:

```

Cluster hasil Agglomerative:
[0 0 0 1 0 0 0 0 0 0 1 0 0 0 1 0 1 0 1 1]

```

Kode percobaan (Cell 26):

```

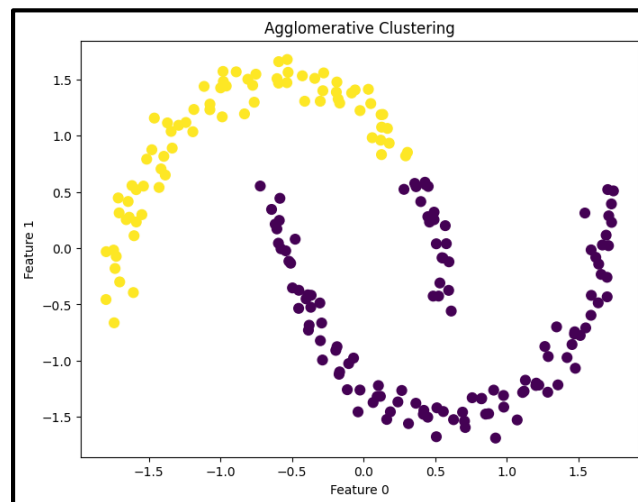
plt.figure(figsize=(8, 6))

plt.scatter(
    X_scaled[:, 0],
    X_scaled[:, 1],
    c=clusters,
    s=60
)

plt.xlabel("Feature 0")
plt.ylabel("Feature 1")
plt.title("Agglomerative Clustering")

plt.show()

```



Gambar hasil percobaan pada Cell 26.

3.14 DBSCAN dan Pengaruh Parameter

DBSCAN diuji dengan beberapa konfigurasi parameter. Pada konfigurasi awal dan perubahan `min_samples`, jumlah label cluster berubah. Ketika `eps` diatur menjadi 15 dengan

min_samples=3, seluruh 569 data masuk ke satu cluster (label 0). Konfigurasi eps=7 pada bagian yang sama juga menghasilkan satu cluster berisi 569 data. Percobaan ini menunjukkan bahwa parameter eps dan min_samples dapat memengaruhi struktur cluster yang terbentuk.

Kode percobaan (Cell 27):

```
from sklearn.cluster import DBSCAN
import numpy as np

# DBSCAN dengan parameter default
dbscan = DBSCAN()

labels = dbscan.fit_predict(X_pca)

print("Unique labels:", np.unique(labels))
```

Output:

```
Unique labels: [-1  0  1  2  3  4  5  6]
```

Kode percobaan (Cell 28):

```
dbscan = DBSCAN(min_samples=3)

labels = dbscan.fit_predict(X_pca)

print("Unique labels:", np.unique(labels))
```

Output:

```
Unique labels: [-1  0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22
 23]
```

Kode percobaan (Cell 29):

```
dbscan = DBSCAN(min_samples=3, eps=15)

labels = dbscan.fit_predict(X_pca)

print("Unique labels:", np.unique(labels))
```

Output:

```
Unique labels: [0]
```

Kode percobaan (Cell 30):

```
print(
    "Number of points per cluster:",
    np.bincount(labels + 1)
)
```

Output:

```
Number of points per cluster: [  0 569]
```

Kode percobaan (Cell 31):

```
dbscan = DBSCAN(min_samples=3, eps=7)

labels = dbscan.fit_predict(X_pca)
```

```
print("Unique labels:", np.unique(labels))
print("Number of points per cluster:", np.bincount(labels + 1))
```

Output:

```
Unique labels: [0]
Number of points per cluster: [ 0 569]
```

3.15 DBSCAN pada Data Wajah Setelah PCA

Data wajah dengan ukuran awal 2267×5655 direduksi menggunakan PCA menjadi 2267×100 dengan whitening. DBSCAN kemudian dijalankan menggunakan `min_samples=3` dan `eps=7`.

Output menunjukkan label -1 sebagai noise dan beberapa cluster kecil. Jumlah data yang terdeteksi sebagai noise adalah 2115, sedangkan cluster lain memiliki ukuran kecil. Hasil ini menunjukkan bahwa konfigurasi parameter yang digunakan menghasilkan banyak data yang dianggap sebagai noise.

Kode percobaan (Cell 32):

```
from sklearn.decomposition import PCA
from sklearn.cluster import DBSCAN
import numpy as np
import matplotlib.pyplot as plt

# PCA pada X_train
pca = PCA(n_components=100, whiten=True, random_state=0)
X_train_pca = pca.fit_transform(X_train)

print("X_train:", X_train.shape)
print("X_train_pca:", X_train_pca.shape)
```

Output:

```
X_train: (2267, 5655)
X_train_pca: (2267, 100)
```

Kode percobaan (Cell 33):

```
# DBSCAN
dbscan = DBSCAN(min_samples=3, eps=7)

labels = dbscan.fit_predict(X_train_pca)

print("Unique labels:", np.unique(labels))
print("Number of points per cluster:", np.bincount(labels + 1))
```

Output:

```
Unique labels: [-1  0  1  2  3  4  5  6  7  8  9 10]
Number of points per cluster: [2115  3 115  3  4  3  4  8  3  3  3  3]
```

3.16 Perbandingan K-Means, Agglomerative, dan DBSCAN menggunakan ARI

Pada dataset `make_moons`, K-Means, Agglomerative Clustering, dan DBSCAN dibandingkan menggunakan Adjusted Rand Index (ARI). Notebook juga membuat random assignment sebagai pembanding. ARI digunakan untuk mengukur kesesuaian hasil cluster terhadap label yang

tersedia pada dataset. Grafik hasil perbandingan memperlihatkan perbedaan pola pengelompokan yang dihasilkan oleh masing-masing algoritma.

Kode percobaan (Cell 35):

```
from sklearn.datasets import make_moons
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans, AgglomerativeClustering, DBSCAN
from sklearn.metrics.cluster import adjusted_rand_score
import numpy as np
import matplotlib.pyplot as plt
```

Kode percobaan (Cell 36):

```
X, y = make_moons(
    n_samples=200,
    noise=0.05,
    random_state=0
)

# Rescale data
scaler = StandardScaler()
scaler.fit(X)
X_scaled = scaler.transform(X)
```

3.17 Perbandingan menggunakan Silhouette Score

Silhouette Score digunakan untuk mengevaluasi kualitas pemisahan cluster pada dataset make_moons. Nilai yang ditampilkan pada grafik digunakan untuk membandingkan struktur cluster yang dihasilkan oleh random assignment, K-Means, Agglomerative Clustering, dan DBSCAN. Nilai silhouette yang lebih tinggi secara umum menunjukkan struktur cluster yang lebih terpisah pada data yang sedang dievaluasi.

Kode percobaan (Cell 37):

```
fig, axes = plt.subplots(
    1, 4,
    figsize=(15, 3),
    subplot_kw={'xticks': (), 'yticks': ()}
)

# Algoritma yang digunakan
algorithms = [
    KMeans(n_clusters=2),
    AgglomerativeClustering(n_clusters=2),
    DBSCAN()
]

# Random cluster assignment
random_state = np.random.RandomState(seed=0)
random_clusters = random_state.randint(
    low=0,
    high=2,
    size=len(X)
)

# Random assignment
```

```

axes[0].scatter(
    X_scaled[:, 0],
    X_scaled[:, 1],
    c=random_clusters,
    s=60
)

axes[0].set_title(
    "Random assignment - ARI: {:.2f}".format(
        adjusted_rand_score(y, random_clusters)
    )
)

# KMeans, Agglomerative, DBSCAN
for ax, algorithm in zip(axes[1:], algorithms):

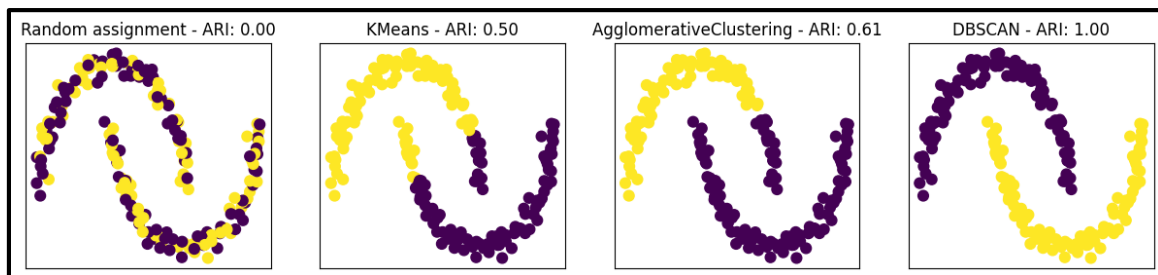
    clusters = algorithm.fit_predict(X_scaled)

    ax.scatter(
        X_scaled[:, 0],
        X_scaled[:, 1],
        c=clusters,
        s=60
    )

    ax.set_title(
        "{} - ARI: {:.2f}".format(
            algorithm.__class__.__name__,
            adjusted_rand_score(y, clusters)
        )
    )

plt.show()

```



Gambar hasil percobaan pada Cell 37.

3.18 Rekonstruksi Citra dengan K-Means, PCA, dan NMF

Pada bagian ini digunakan 100 komponen untuk NMF dan PCA serta 100 cluster untuk K-Means. Data test direkonstruksi menggunakan ketiga pendekatan tersebut. Hasil visualisasi menampilkan komponen yang diekstrak serta perbandingan citra asli dengan citra hasil rekonstruksi. Bagian ini memperlihatkan bahwa reduksi dimensi dan clustering dapat digunakan untuk membentuk representasi data yang lebih ringkas.

Kode percobaan (Cell 38):

```

from sklearn.metrics.cluster import silhouette_score

```

Kode percobaan (Cell 39):

```
X, y = make_moons(
    n_samples=200,
    noise=0.05,
    random_state=0
)

# Rescale data
scaler = StandardScaler()
scaler.fit(X)
X_scaled = scaler.transform(X)
```

Kode percobaan (Cell 40):

```
fig, axes = plt.subplots(
    1, 4,
    figsize=(15, 3),
    subplot_kw={'xticks': (), 'yticks': ()}
)

# Random assignment
random_state = np.random.RandomState(seed=0)

random_clusters = random_state.randint(
    low=0,
    high=2,
    size=len(X)
)

axes[0].scatter(
    X_scaled[:, 0],
    X_scaled[:, 1],
    c=random_clusters,
    s=60
)

axes[0].set_title(
    "Random assignment: {:.2f}".format(
        silhouette_score(X_scaled, random_clusters)
    )
)

# Algoritma clustering
algorithms = [
    KMeans(n_clusters=2),
    AgglomerativeClustering(n_clusters=2),
    DBSCAN()
]

for ax, algorithm in zip(axes[1:], algorithms):

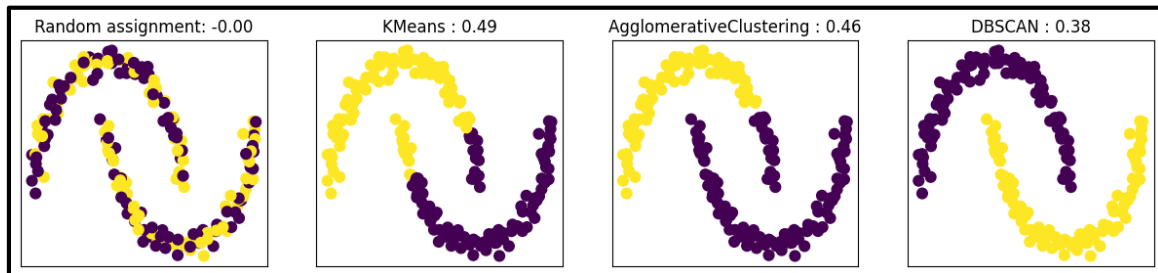
    clusters = algorithm.fit_predict(X_scaled)

    ax.scatter(
        X_scaled[:, 0],
        X_scaled[:, 1],
        c=clusters,
        s=60
    )
```

```

ax.set_title(
    "{} : {:.2f}".format(
        algorithm.__class__.__name__,
        silhouette_score(X_scaled, clusters)
    )
)
plt.show()

```



Gambar hasil percobaan pada Cell 40.

Kode percobaan (Cell 41):

```

from sklearn.decomposition import NMF, PCA
from sklearn.cluster import KMeans
from sklearn.model_selection import train_test_split
import numpy as np
import matplotlib.pyplot as plt

# Membagi data menjadi data training dan testing
X_train, X_test, y_train, y_test = train_test_split(
    X_people, y_people,
    stratify=y_people,
    random_state=0
)

# NMF dengan 100 komponen
nmf = NMF(n_components=100, random_state=0)
nmf.fit(X_train)

# PCA dengan 100 komponen
pca = PCA(n_components=100, random_state=0)
pca.fit(X_train)

# K-Means dengan 100 cluster
kmeans = KMeans(n_clusters=100, random_state=0)
kmeans.fit(X_train)

# Rekonstruksi data menggunakan PCA
X_reconstructed_pca = pca.inverse_transform(
    pca.transform(X_test)
)

# Rekonstruksi data menggunakan K-Means
X_reconstructed_kmeans = kmeans.cluster_centers_[
    kmeans.predict(X_test)
]

# Rekonstruksi data menggunakan NMF
X_reconstructed_nmf = np.dot(
    nmf.transform(X_test),

```

```
nmf.components_  
)
```

Output:

```
/usr/local/lib/python3.13/dist-packages/sklearn/decomposition/_nmf.py:1742:  
ConvergenceWarning: Maximum number of iterations 200 reached. Increase it to improve  
convergence.  
warnings.warn(  
/usr/local/lib/python3.13/dist-packages/sklearn/decomposition/_nmf.py:1742:  
ConvergenceWarning: Maximum number of iterations 200 reached. Increase it to improve  
convergence.  
warnings.warn(  

```

3.19 Perbandingan Agglomerative dan K-Means

Agglomerative Clustering dan K-Means masing-masing digunakan untuk membentuk 10 cluster pada representasi PCA. Ukuran cluster Agglomerative adalah [85, 71, 165, 39, 15, 12, 94, 53, 5, 30], sedangkan K-Means menghasilkan [115, 16, 84, 76, 49, 68, 30, 13, 113, 5]. Perbandingan label kedua metode menggunakan Adjusted Rand Index menghasilkan nilai 0,47. Nilai tersebut menunjukkan adanya kesamaan tertentu antara dua pembagian cluster, tetapi hasil kedua metode tidak identik.

Kode percobaan (Cell 44):

```
from sklearn.cluster import AgglomerativeClustering  
import numpy as np  
  
# extract clusters with ward agglomerative clustering  
agglomerative = AgglomerativeClustering(n_clusters=10)  
  
labels_agg = agglomerative.fit_predict(X_pca)  
  
print("Cluster sizes agglomerative clustering: {}".format(  
    np.bincount(labels_agg)  
))
```

Output:

```
Cluster sizes agglomerative clustering: [ 85  71 165  39  15  12  94  53   5  30]
```

Kode percobaan (Cell 45):

```
from sklearn.cluster import KMeans  
  
# K-Means dengan 10 cluster  
km = KMeans(n_clusters=10, random_state=0)  
  
# Membuat label cluster  
labels_km = km.fit_predict(X_pca)  
  
print("Cluster sizes k-means: {}".format(  
    np.bincount(labels_km)  
))
```

Output:

```
Cluster sizes k-means: [115  16  84  76  49  68  30  13 113   5]
```

Kode percobaan (Cell 46):

```
from sklearn.metrics.cluster import adjusted_rand_score

print("ARI: {:.2f}".format(
    adjusted_rand_score(labels_agg, labels_km)
))
```

Output:

ARI: 0.47

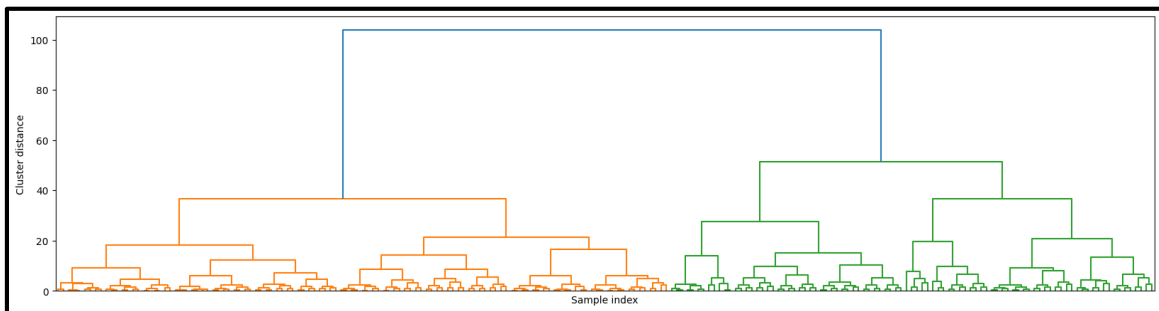
Kode percobaan (Cell 47):

```
from scipy.cluster.hierarchy import dendrogram, ward
import matplotlib.pyplot as plt

linkage_array = ward(X_pca)

plt.figure(figsize=(20, 5))
dendrogram(
    linkage_array,
    p=7,
    truncate_mode='level',
    no_labels=True
)

plt.xlabel("Sample index")
plt.ylabel("Cluster distance")
plt.show()
```



Gambar hasil percobaan pada Cell 47.

3.20 Agglomerative Clustering dengan 10 dan 40 Cluster serta Silhouette

Pada data yang dipilih sebanyak 569 sampel, PCA menghasilkan 100 komponen. Agglomerative Clustering dengan 10 cluster menghasilkan ukuran cluster [40, 54, 41, 104, 44, 87, 26, 75, 35, 63]. Ketika jumlah cluster diubah menjadi 40, ukuran cluster menjadi lebih kecil dan lebih banyak. Pada konfigurasi 40 cluster, Silhouette Score yang tercatat adalah 0,02. Nilai tersebut menunjukkan bahwa struktur cluster pada konfigurasi ini memiliki pemisahan yang lemah, sehingga pembagian menjadi 40 cluster tidak menghasilkan kelompok yang sangat terpisah berdasarkan metrik tersebut.

Kode percobaan (Cell 48):

```

from sklearn.decomposition import PCA
from sklearn.cluster import AgglomerativeClustering
import numpy as np
import matplotlib.pyplot as plt

# Gunakan 569 data yang sama dengan X_pca sebelumnya
X_cluster = X_train[:569]
y_cluster = y_train[:569]

# PCA
pca = PCA(n_components=100, random_state=0)
X_pca = pca.fit_transform(X_cluster)

print("Bentuk X_cluster:", X_cluster.shape)
print("Bentuk X_pca:", X_pca.shape)

# Agglomerative Clustering
agglomerative = AgglomerativeClustering(n_clusters=10)
labels_agg = agglomerative.fit_predict(X_pca)

print("Cluster sizes agglomerative clustering: {}".format(
    np.bincount(labels_agg)
))

```

Output:

```

Bentuk X_cluster: (569, 5655)
Bentuk X_pca: (569, 100)
Cluster sizes agglomerative clustering: [ 40  54  41 104  44  87  26  75  35  63]

```

Kode percobaan (Cell 49):

```

n_clusters = 10

for cluster in range(n_clusters):
    mask = labels_agg == cluster

    fig, axes = plt.subplots(
        1, 10,
        subplot_kw={'xticks': (), 'yticks': ()},
        figsize=(15, 8)
    )

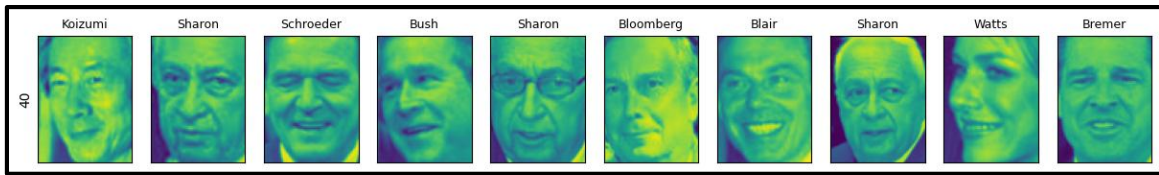
    axes[0].set_ylabel(np.sum(mask))

    images = X_cluster[mask]
    labels = y_cluster[mask]

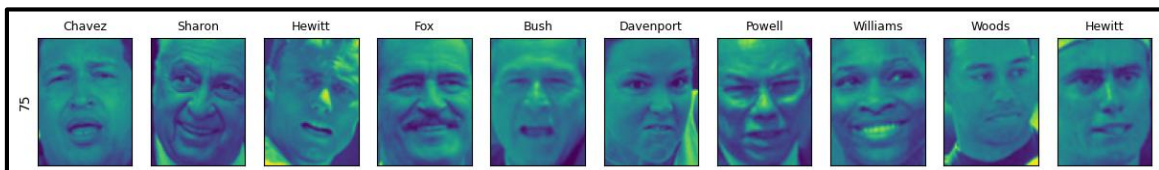
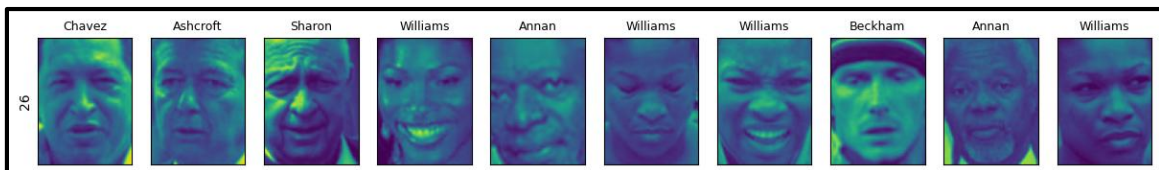
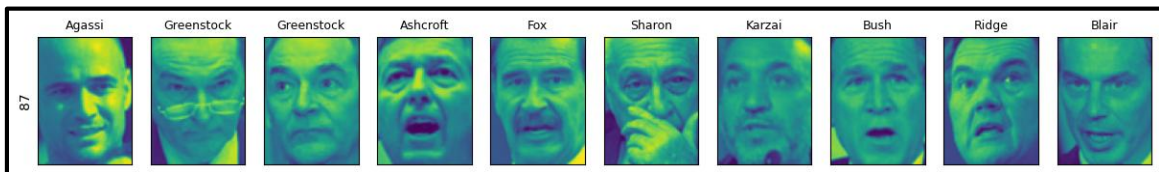
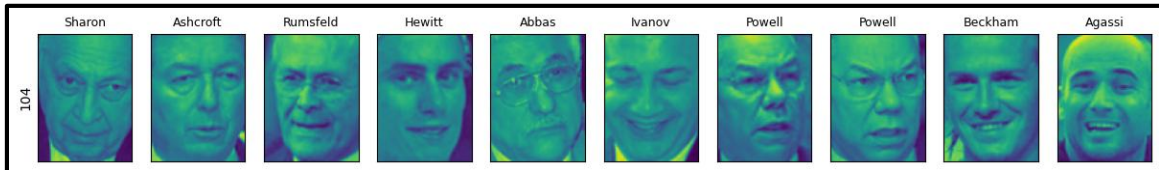
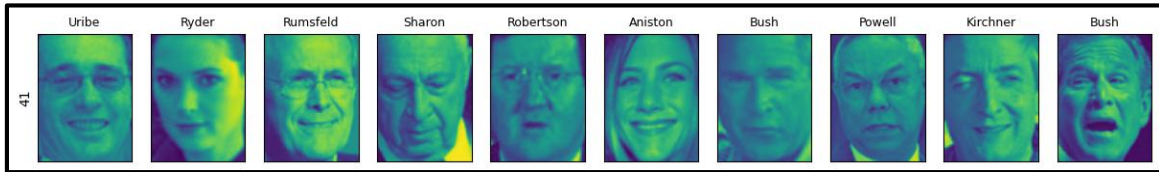
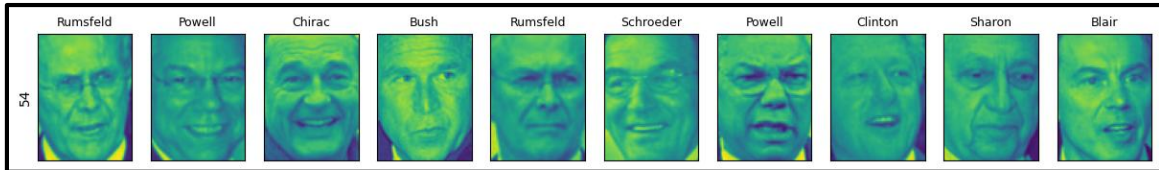
    for image, label, ax in zip(images, labels, axes):
        ax.imshow(
            image.reshape(image_shape),
            vmin=0,
            vmax=1
        )
        ax.set_title(
            people.target_names[label].split()[-1],
            fontdict={'fontsize': 9}
        )

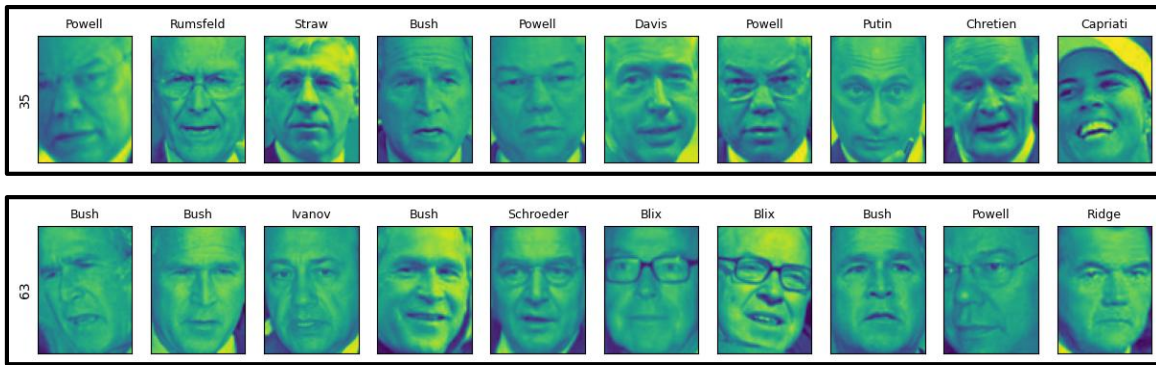
plt.show()

```



Gambar hasil percobaan pada Cell 49.





Kode percobaan (Cell 50):

```
agglomerative = AgglomerativeClustering(n_clusters=40)

labels_agg = agglomerative.fit_predict(X_pca)

print("Cluster sizes agglomerative clustering: {}".format(
    np.bincount(labels_agg)
))
```

Output:

```
Cluster sizes agglomerative clustering: [ 9 19 11 25 26 24  8 20 15 20 28 19 16 10  4 21 13  7
 27  6  7  9  5 14
 13 18  5 15  5 12 15 11 27  3 17 17 21  7  6 14]
```

Kode percobaan (Cell 52):

```
from sklearn.metrics.cluster import silhouette_score

print("Silhouette Score: {:.2f}".format(
    silhouette_score(X_pca, labels_agg)
))
```

Output:

```
Silhouette Score: 0.02
```

BAB IV

KESIMPULAN

Berdasarkan seluruh percobaan, beberapa algoritma klasifikasi dapat diterapkan pada dataset forge dan menghasilkan nilai akurasi yang berbeda. KNN, Naive Bayes, dan MLP menghasilkan akurasi 85,71%, sedangkan Random Forest menghasilkan 71,43%. Decision Tree memperoleh 100% pada cara penilaian yang digunakan dalam notebook, yaitu pada keseluruhan data, sehingga hasil tersebut perlu dibaca dengan memperhatikan metode evaluasinya.

Pada tahap preprocessing dan reduksi dimensi, MinMax Scaling berhasil mengubah rentang fitur menjadi 0 sampai 1. PCA pada dataset breast cancer mengurangi 30 fitur menjadi dua komponen dengan total explained variance sekitar 63,24%. NMF dan t-SNE juga menunjukkan bagaimana data berdimensi tinggi dapat direpresentasikan dalam bentuk yang lebih ringkas untuk analisis dan visualisasi.

Pada clustering, K-Means, Agglomerative Clustering, dan DBSCAN menghasilkan struktur cluster yang berbeda. Perubahan parameter DBSCAN dapat mengubah jumlah cluster dan jumlah data yang dianggap noise. Perbandingan Agglomerative dan K-Means menghasilkan ARI sebesar 0,47, sedangkan konfigurasi Agglomerative dengan 40 cluster menghasilkan Silhouette Score 0,02. Hasil tersebut menunjukkan pentingnya pemilihan algoritma, parameter, dan metrik evaluasi ketika melakukan clustering.